

Activity 08 - JSON contracts, enum parsing, regex and file lifetime

AI Vibe Coding with C# | TGS-2023039178 | v7.0 | 13 September 2026

Alignment: LO4 | K4 | A4

Suggested time: 45 minutes.

A warehouse imports synthetic product data using a documented JSON contract. Reject mismatched types and negative stock, preserve UTF-8 names and prove export/import round-trip interoperability.

Prerequisites

Install the .NET 10 SDK for your Windows/macOS/Linux machine and VS Code. C# Dev Kit is optional for debugging. Use `dotnet --list-sdks` to confirm a 10.x SDK. All sample data are synthetic. An AI subscription is optional; every activity works without AI and without API keys. The classroom programme includes guided demonstrations before this independent task.

Files

- Activity.csproj: compile settings.
- Program.cs: runnable teaching example and acceptance checks.
- requirements.md: business specification and extension tasks.
- prompts.md: bounded AI review prompts.
- expected-output.txt: baseline demonstration output.
- evidence.md: learner evidence checklist.

Detailed procedure

1. Open this activity folder in VS Code. Read requirements.md and identify the input, output, invariant and error path before editing.
2. Open a terminal in this folder and verify an SDK is installed. An SDK is required; a runtime alone cannot compile the project.

```
dotnet --list-sdks
```

3. Read Activity.csproj. Confirm net10.0, nullable reference checking and warnings-as-errors. Read Program.cs and locate the business rule named in the requirements.
4. Build the supplied project and resolve every compiler error before running it.

```
dotnet build Activity.csproj
```

5. Run the deterministic demonstration. Compare each line to expected-output.txt; record the actual output in your evidence checklist.

```
dotnet run --project Activity.csproj
```

6. Run the in-program acceptance checks. Read the checks in Program.cs; they call real production methods and must finish without an exception.

```
dotnet run --project Activity.csproj -- --self-test
```

7. Make a baseline copy of Program.cs outside this activity, then complete the extension tasks below. Explain each change in your own words; keep the code runnable.
8. Ask an approved AI assistant for a review using prompts.md, or perform the same review manually. Share only the synthetic sample, specification and error excerpt. Inspect the proposed diff before applying it.
9. Rebuild, rerun the normal/invalid/boundary tests and verify that the change preserves the invariant. If it fails, record the failing input, expected/actual output and smallest repair.

```
dotnet build Activity.csproj
dotnet run --project Activity.csproj -- --self-test
```

10. Complete the evidence record with source file names, actual outputs, one failure/repair, the requirement mapping and a short explanation of why the chosen controls fit.

Extension tasks

- Create a contract table with field, type, required value, valid example and failure.
- Add missing-field and JSON-null tests and document serializer defaults.
- Add a schema-version field and explain backward compatibility.
- Explain why using `close` on `StreamWriter` deterministically and a `finalizer` is unsuitable.

Acceptance criteria

- A valid record survives JSON export/import unchanged.
- Quantity is a JSON integer, not text; stock cannot be negative.
- SKU uses exactly SKU- plus three digits.
- UTF-8 names and zero stock are preserved.

Expected baseline output

```
Imported SKU-001; stock 10
Round-trip equal: True
Export: output/product.json
```

Troubleshooting

- NETSDK1045: install/select .NET 10 SDK; do not silently retarget the project.
- CS...: read the first compiler error, fix the referenced line and rebuild. Nullable warnings identify a missing null check.
- Wrong folder/project: open the terminal beside Activity.csproj and use the explicit `--project` option.
- Failed assertion: compare the test input, expected value and production rule before changing either.
- A generated file cannot be written: check the current folder permissions; use the local output/ folder and close open file handles.
- AI proposes C++ syntax, a new package or destructive file operations: reject that portion and request a C#/NET-only diff within the existing project.

References

[Microsoft C# documentation](#) | [Microsoft beginner training](#)